00 — Competition Brief

This document answers: what the competition is, what is scored, what is provided, what the rules allow, and what "winning" looks like as of **2026-10-24**. It is the orientation layer; the technical contract is in `01-harness-and-evaluation.md`.

---

1. The premise

Today's strongest coding agents depend on large cloud models and internet access. The competition asks whether a **Gemma 4 model running offline on consumer-grade accelerators** can navigate a real codebase, understand an issue, and produce a patch that passes hidden tests (S1).

You are not training a model from scratch. The base model is fixed to one of four Gemma 4 QAT W4A16 variants (S1). You control the **agent scaffold**, the **prompt**, the **tool policy**, the **budgets**, optional **skills**, and optional **LoRA adapters** (S5:57,191-201). The competition is therefore mostly an agent-engineering and post-training problem, not a pretraining problem.

The host's stated goal is to accelerate research in autonomous coding agents and make offline developer agents practical (S1); the Gemma 4 technical report provides the model-family background (S9). The paper track explicitly invites research on tuning, code comprehension, benchmarks, and graph reasoning (S46).

---

2. Tracks, prizes, and dates

Track	Prize	Notes
Main competition	**$65,000**	1st $37k, 2nd 18k, 3rd $10k (S1)
Small-model bonus	**$10,000**	Best E2B/E4B entry; must be one of two final submissions; LoRA allowed; cross-eligible (S47)

Track	Prize	Notes
Paper track	**$35,000**	Best paper $15k$, $best new resource$10k, best new application $10k (S46)
Total	**$110,000**	Main + small-model + paper (S1)

Date (UTC)	Event
2026-09-23	Competition start (S1)
2026-11-12	Paper-track deadline (S46)
2026-11-25	Entry deadline and team-merger deadline (S1)
2026-12-02	Final submission deadline (S1)

All deadlines are 11:59 PM UTC unless noted (S1). The paper track is a separate Kaggle competition; joining it is separate from the main track (S46). The small-model bonus is part of the main competition but requires an eligible E2B/E4B base model and a final selection slot (S47).

At retrieval, the main competition had **12,493 entrants, 3,959 participants, 3,548 teams, and 22,065 submissions** (S1). The paper track had **5,620 entrants, 338 participants, 327 teams, and 348 submissions** (S46). The paper track is much less crowded, but judging is subjective and requires a real research contribution.

---

3. What is scored

The score is the **resolution rate** over the hidden test set:

$$\text{score} = \frac{\#\{\text{tasks resolved}\}}{\#\{\text{tasks evaluated}\}}$$

A task is resolved only if:

1. the agent's patch applies to a fresh baseline,
2. the hidden `test_patch` applies,
3. pytest exits 0,
4. the JUnit XML has `passed_tests > 0`, `failures == 0`, `errors == 0`, and every required test node passed without being skipped.

This is stricter than “pytest exited 0.” A patch that causes tests to be skipped, or that deletes the required test, does not count (S5:565-567). There is no partial credit per task.

The agent has **12 hours total** for patch generation across all tasks, inclusive of sandbox setup, excluding patch validation (S1). Participants report that exceeding the global limit produces a timeout with no partial score (S29). Per-task budgets are separate and optional; they are safety valves, not the global limit (S5:493-509).

The public leaderboard is calculated with approximately **50%** of the test data; final results use the other 50% (S4). At retrieval, the top public score was **0.24**, with a large cluster at **0.22** (S4). Terminology: the **public training set** is the 129 tasks you can download; the **public LB split** is the ~58 hidden tasks used for the public leaderboard; the **final/private split** is the other ~60 hidden tasks. One task is about 1/58 ≈ 0.017 on the public LB and about 1/60 ≈ 0.017 on the final split; 1/120 ≈ 0.0083 is only the blended rate over all hidden tasks. Displayed scores are truncated to two decimals (S4, S39).

---

4. What is provided

The public dataset is **22.42 GB** and contains (S2):

- **129 public training tasks** from `fastapi/fastapi`, `Textualize/rich`, `psf/requests`, and `encode/httpx`.
- `tasks.jsonl`: `instance_id`, `repo`, `base_commit`, `problem_statement`, `hints_text`, `patch`, `test_patch`, `created_at`.
- `snapshots/`: 129 Git working trees at `base_commit`, with future history removed.
- `graphs/` and `embeddings/`: AST call/dependency graphs and 256-d node embeddings for the public repos.
- `wheels/`: 124 precompiled offline wheels for historical dependency versions.
- `docker/`, `sandbox/setup.py`, `sample_submission/`, `HARNESS_README.md`, and the `submission.parquet` schema. You upload `submission.zip`; the scorer writes `[id, prediction]` to `submission.parquet` (S5:61,537). You do not produce the parquet.

The hidden test set is about **120 tasks**, evenly divided between public and private splits, curated from **private repositories** (S2). The host says no public training

repository appears in the test set, and every hidden task validates 100% with a gold patch (S31). The test set was filtered with a frontier-model **near-completion** check and for fail-to-pass/pass-to-pass behavior (S2); the separate “gold patch validates 100%” claim comes from the host (S31), not from the near-completion filter.

The public tasks are for development and local evaluation. They are not the hidden test set, and they are not a random sample of it: they come from four popular open-source repos, while the hidden tasks come from private repos. Local scores on public tasks are therefore a weak predictor of hidden scores (S31, S35, S39).

---

5. What the agent sees

For each task, the harness creates a sandbox at `/workspace` containing the repository snapshot at `base_commit`. The agent receives (S5:282-319):

- the repository name and problem statement,
- optional hints,
- the task budget (time, tool calls, turns, cost),
- execution environment rules (300 s command timeout, 5,000-char output, 150 lines/10,000 chars per `read_file`, offline),
- standard instructions to work under `/workspace`, inspect conventions, verify with targeted tests, and call `submit_patch`,
- code-intelligence guidance if graph tools are declared and graph files exist,
- a workspace layout of up to 200 entries.

The agent does **not** see the hidden `test_patch` or the hidden tests. It may see existing tests in the repository and can run them. It must fix the source code, not the tests (S5:535,549,556).

The environment is air-gapped: no internet, no PyPI (S5:235). Dependencies are preinstalled from the wheelhouse. Commands are limited to 300 s and 5,000 chars of output; the container has 4 GiB RAM and 2 vCPUs (S5:233-242). Heavy full-suite test runs are a bad idea.

---

6. The four allowed models

Only these base models are allowed in a competition submission (S1, S5:155-173):

Model	Approx. weights on 4×L4	Practical role
`gemma-4-31b-it-qat-w4a16-ct`	~16–18 GB total	Best empirical quality; default choice (S34)
`gemma-4-12b-it-qat-w4a16-ct`	~6–8 GB	Faster decode, but writes more and solves fewer tasks (S34)
`gemma-4-e4b-it-qat-w4a16-ct`	~3–4 GB	Small-model bonus; weaker on hard tasks (S34, S47)
`gemma-4-e2b-it-qat-w4a16-ct`	~2–3 GB	Small-model bonus; weakest (S47)

All agents in a submission must declare the **same** base model (S5:170-173). Different agents may use different LoRA adapters, as long as the base model is the same (S5:191-201).

The model family is Gemma 4, released April 2026 (S48), with QAT checkpoints released June 2026 (S49). The competition uses the QAT W4A16 compressed-tensors variants for vLLM (S7, S43). These are inference checkpoints; they are not directly trainable with PEFT (S25, S45). The unquantized QAT checkpoint (`-qat-q4_0-unquantized`) is the intended training/research starting point (S7, S43).

---

7. What the public leaderboard says

At retrieval (S4):

- Top public score: **0.24** (four teams).
- Large cluster: **0.22**.
- Then **0.20**.
- The public leaderboard is ~50% of the test data; final standings use the private 50%.

The score distribution is compressed because the base model, context window, and task set are fixed. A few tasks separate the top teams. Community reports show substantial run-to-run variance:

- [COMMUNITY] Identical agent files scored ~0.12–0.22 public in one study (S34).
- The same byte-identical zip scored 0.10 and 0.15 in one observed period (S39).
- A public entry's local 28/74 screening produced hidden scores of 0.17 and 0.10 for tied configs (S35).
- A public prompt-only notebook *claims* 0.24 (S36; author-reported); it is a strong baseline, not proof that post-training is required.

This has two implications:

1. **Do not overfit to a single score.** Use paired local comparisons and multiple submissions.
2. **Submit early.** A known-good 0.15–0.20 bundle is more valuable than an unsubmitted 0.24 idea, especially with queue delays (S30).

---

8. Rules that affect strategy

The full rules are at S3. The strategic points:

- **One submission per day; up to two final submissions** (S3). If fewer than two are selected, Kaggle auto-selects from your best scoring submissions (S1).
- **Team size max 5; team merger deadline 2026-11-25** (S3).
- **External data is allowed if it is publicly available and equally accessible at no cost, or meets the Reasonableness Standard** (S3). If you train on trajectories, plan to publish the dataset or use public data.
- **External models are allowed unless specifically prohibited by the Host** (S3). Check the licenses and the Reasonableness Standard; a proprietary dataset that costs more than a prize is not reasonable.
- **Winners must provide training code, inference code, and a reproducible methodology** (S3). Keep experiment manifests, configs, and hashes from day one.
- **Winning submission and source code are licensed Apache 2.0** (S3). Do not build on code/data you cannot license.
- **Competition Data may not be redistributed to non-participants** (S3). Do not publish the hidden test set or private task data; public training tasks have their own licenses.

If you use public notebooks or code, check their licenses. The public prompt-only notebooks are Apache 2.0 (S36, S37); the public entry repo is Apache 2.0 (S35).

Attribution is good practice and may be required by the license.

License inventory (do this from day 1):

Artifact	What to check
Gemma 4 weights/adapters	Gemma 4 license/Apache 2.0 model card (S8)
Public task snapshots	Competition rules; each repo's `LICENSE` at `base_commit`
Trajectory dataset	Your generation code + source repo licenses; derivative-work question
Prompts/configs/code	Your chosen license (Apache 2.0 recommended for winner compliance)
Public notebooks used as references	Their stated license (S36/S37 Apache 2.0)

Do not assert a repo license without checking its `LICENSE` at the task's `base_commit`. If you cannot state an artifact's license in one line, do not publish it.

---

9. What "winning" looks like

The main track pays three places. At retrieval, 0.24 was the top public score and 0.22 was a large cluster (S4). A realistic progression for a new team:

Stage	Public score (rough)	What it means
Valid submission	any score	Pipeline works; patch is non-empty; tests run
Early baseline	0.06–0.12	Agent edits and submits, but localization/verification is weak (S40)
Competitive	0.15–0.20	Good prompt, budgets, and patch discipline
Contender	0.22–0.24	Strong scaffold and/or adapter; top of public LB

These are **planning bands, not measured thresholds**. The private split can move standings; a public 0.22 may not hold private (S4, S39). There is no guaranteed

“winner” score.

The small-model bonus is a separate lane: the best small-model public score was ~0.06 as of 2026-10-20 (S47). A well-engineered E4B agent could be competitive for $10k even if it is far from the main leaderboard. The paper track is a third lane: $327 teams and $35k, judged on research quality (S46). If you have limited compute or strong research skills, the paper track may be the best expected-value path.

---

10. What this competition is really testing

Based on the harness and public evidence, the competition rewards:

1. **Patch delivery.** An agent that never edits scores zero, regardless of model quality. The public 0.18 analysis found runs with no source edit by call 12 fail 87% of the time (S38).
2. **Budget discipline.** The 12h global limit and per-task caps reward agents that stop exploring and start editing (S29, S38).
3. **Localization.** The hidden tests are written from the issue text; exact names, messages, types, and edge cases matter (S36).
4. **Patch hygiene.** Test edits are stripped; scratch files pollute the patch; untracked files are included via `git add -N` (S5:535,625).
5. **Evaluation discipline.** Local scores are noisy and environment-sensitive; teams that calibrate gold patches and use paired comparisons make better decisions (S31, S32, S39).
6. **Risk management.** Queue delays, vLLM startup failures, LoRA KV-cache issues, and global timeouts can lose a submission (S23–S30).

The model matters, but the public evidence suggests the scaffold and budget matter more at the margin. The 31B model is the best base (S34), but a prompt-only 31B notebook claims 0.24 public (S36; author-reported). Post-training must beat a strong scaffold, not a weak one.